HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering



DEEP LEARNING AND ITS APPLICATIONS

Transfer Learning for Fine-Grained Food Recognition

Fine-Tuning and Compute Trade-offs on Food-101

Instructor: Le Thanh Sach, Ph.D.
Course: CO3133 – Semester 261
Group: MobyDick

Student Name	Student ID	Contribution
Le Quoc Binh	2352117	Custom CNN, ResNet-50 tuning
Quach Kien Long	2352698	Data/audit, EDA, preprocessing
Le Thanh Lam	2252421	Experiments, AMP & profiling
To Nha Vy	2353360	Errors, Grad-CAM, report/pages

Ho Chi Minh City, November 2026

Contents

1 Introduction **2**

2 Datasets and Preprocessing **2**

2.1 Dataset Description and Scope 2

2.2 Exploratory Data Analysis and Integrity Audit 3

2.3 Data Partitioning and Leakage Prevention 5

2.4 Preprocessing and Augmentation 5

3 Methodology **6**

3.1 Pipeline Overview 6

3.2 Model Architectures 6

3.3 Backbone Update Policies 8

4 Experimental Protocol **8**

4.1 Training Configuration and Checkpoint Selection 8

4.2 Experimental Design 9

4.3 Hardware and Software Environment 10

4.4 Reproducibility and Measurement Protocol 10

5 Results and Discussion **11**

5.1 Training Dynamics 11

5.2 Food-101 Benchmark 12

5.3 Augmentation Ablation 14

5.4 Qualitative Results and Interpretability 15

5.5 Error Analysis 18

5.6 Limitations 19

6 Conclusion **19**

1 Introduction

Fine-grained food recognition supports dietary assessment, nutritional logging and automated restaurant checkout. The task is difficult because the appearance of a dish changes with plating, viewpoint and illumination, while different categories can share colors, ingredients and surface textures. This report examines transfer learning on a predefined ten-class Food-101 subset, using a custom convolutional network, a pretrained ResNet-50 with frozen backbone parameters, and a two-stage fully fine-tuned ResNet-50.

Given a preprocessed image $X \in \mathbb{R}^{3\times224\times224}$, a model f_θ produces class logits $z = f_\theta(X)$ for $K = 10$ food categories. The prediction unit is an individual image with label $y \in \{0, 1, \ldots, 9\}$. Parameters are optimized using multiclass cross-entropy, and the predicted class is the largest output logit:

$$\mathcal{L}_{\mathrm{CE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\log\left(\frac{\exp(z_{i,y_i})}{\sum_{c=0}^{K-1}\exp(z_{i,c})}\right), \quad \hat{y}_i = \arg\max_{c\in\{0,\ldots,9\}} f_\theta(X_i)_c \tag{1}$$

Research question. Does two-stage full fine-tuning improve recognition sufficiently to justify its additional training time and allocated GPU memory relative to a frozen pretrained backbone and a custom CNN trained from scratch? The comparison considers accuracy and Macro-F1 jointly with parameter count, training duration, warmed-up latency and peak allocated memory. An augmentation ablation examines whether the training transformation policy changes the observed result.

2 Datasets and Preprocessing

2.1 Dataset Description and Scope

Food-101 was introduced by Bossard, Guillaumin and Van Gool. The official dataset contains 101,000 images across 101 categories and is distributed by the ETH Zurich Computer Vision Laboratory. The release supplies 750 training images and 250 manually reviewed test images per category; training images intentionally retain annotation noise and visual artifacts. The full compressed archive is advertised as approximately 5.00 GB and contains JPEG images with official split metadata. The supplied metadata records its download URL and cryptographic checksum, but not the installed dataset footprint.

The approved dataset proposal specifies the first ten categories in alphabetical order, avoiding a subset chosen from observed model performance. This rule yields 10,000 candidate images: 7,500 official training images and 2,500 official test images. After validation partitioning, the 6,750 training images and 10 classes satisfy the coursework minimums of 5,000 training samples and 5 classes. All empirical conclusions are restricted to this subset.

The official dataset page does not state a separate SPDX license. The project uses the archive for academic coursework, respects image copyrights and does not redistribute the dataset archive. The proposal and release information document intended use; this report does not infer additional reuse permissions.

2.2 Exploratory Data Analysis and Integrity Audit

Class distribution is summarized in Table 1. Every category has 675 training, 75 validation and 250 test samples. The imbalance ratio is 1.00, supporting unweighted cross-entropy without a class-frequency correction. Accuracy measures overall correctness, while Macro-F1 gives equal weight to class-specific precision–recall performance.

Table 1: **Predefined Food-101 Subset and Stratified Split**

ID[a]	Category	Train	Validation	Test	Total
1	Apple pie	675	75	250	1,000
2	Baby back ribs	675	75	250	1,000
3	Baklava	675	75	250	1,000
4	Beef carpaccio	675	75	250	1,000
5	Beef tartare	675	75	250	1,000
6	Beet salad	675	75	250	1,000
7	Beignets	675	75	250	1,000
8	Bibimbap	675	75	250	1,000
9	Bread pudding	675	75	250	1,000
10	Breakfast burrito	675	75	250	1,000
Total (10 classes)		**6,750**	**750**	**2,500**	**10,000**

[a] Display IDs 1–10 follow the exported class order; model targets are indexed 0–9. The official 750 training images per class are split 90/10 using seed 42; the 250 official test images per class are held out.

Equal class counts do not imply equal recognition difficulty. Macro-F1 therefore remains informative when class-specific precision and recall differ despite balanced support.

File integrity and channels. The manifest audit covers all $10,000$ candidate files. No unreadable or exact byte-duplicate images were excluded (Excluded $= 0$). The recorded original modes are $9,999$ RGB images and 1 grayscale image; the loader converts every decoded image to three-channel RGB. Channel conversion therefore provides a common model input without implying that every source file was originally RGB. A valid grayscale image is not corrupted; successful decoding also does not establish annotation correctness.

Duplicate screening. SHA-256 digests identify 0 exact duplicates. Equal 64-bit average perceptual hashes flag 17 candidate near-duplicate pairs. These are screening matches rather than manually confirmed duplicates. Average hashing summarizes coarse luminance structure; matching hashes do not establish identical content, and unmatched hashes do not exclude visual similarity. Manifest joins identify 9 pairs within training, 2 between training and validation, 3 between training and test, and 3 within test. The implementation retains these candidates, so image-level splitting does not establish complete separation of visually similar content.

Representative training images in Figure 1 illustrate the appearance variation behind the class labels. The display provides context for differences in dish composition, plating and background rather than an estimate of their prevalence.



Figure 1: Representative training images from each of the 10 predefined Food-101 classes.

The examples show that food categories cannot be separated reliably by a single color or outline. Garnishes, sauce and serving containers may be shared between labels, motivating evaluation of both global pretrained features and features adapted to the selected food categories.

Image geometry is examined separately in Figure 2. Recorded widths range from 287 to 512 pixels and heights from 231 to 512 pixels; both medians are 512. Aspect ratios range from 0.56 to 2.216, with median 1.0 and mean 1.075.

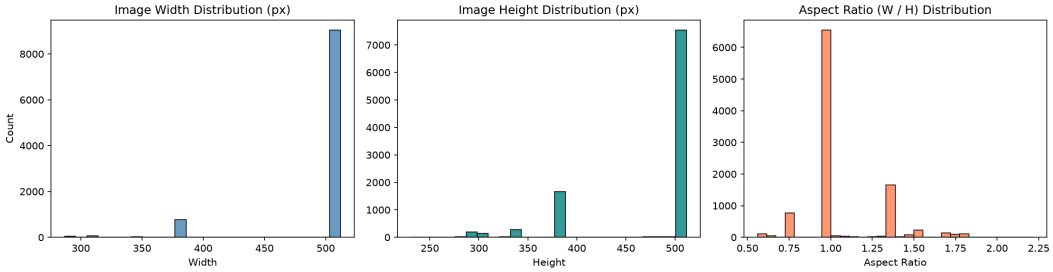


Figure 2: Distributions of image width, height, and aspect ratio (W/H) across the 10,000-image subset.

The maximum side length is 512 pixels, consistent with the dataset release. The observed geometric variation motivates resized cropping during training and aspect-preserving resizing followed by center cropping during evaluation. These operations standardize tensor dimensions but may remove peripheral ingredients or context.

The median aspect ratio does not describe every image: both portrait and landscape examples must be mapped to a common input shape. Cropping can consequently retain different fractions of the original scene. The geometry audit characterizes this preprocessing constraint; it does not quantify how frequently a crop removes a discriminative food component or changes a model prediction.

2.3 Data Partitioning and Leakage Prevention

The 7,500 official training images are split at the image-record level with stratification and seed 42: 90% (6,750 images) for training and 10% (750) for validation. A saved manifest fixes membership across configurations; official test membership is preserved. Validation Macro-F1 selects checkpoints before test evaluation.

The audit covers readability, dimensions and hashes across all partitions. Retained perceptual matches prevent a claim of duplicate-free evaluation; content-group splitting and manual adjudication are undocumented. Test predictions do not determine checkpoint selection.

2.4 Preprocessing and Augmentation

All inputs are converted to RGB tensors and normalized using ImageNet-1K channel statistics, $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$. Table 2 separates the stochastic training branch from deterministic validation and test preprocessing.

Table 2: **Training and Evaluation Preprocessing**

Branch	Ordered transformations	Output[a]
Augmented training	Random resized crop; horizontal flip; color jitter; tensor conversion; normalization	$(3, 224, 224)$
Validation/test	Resize shorter side to 256; center crop 224; tensor conversion; normalization	$(3, 224, 224)$
No-augmentation training	Same deterministic transformations as validation/test	$(3, 224, 224)$

[a] Random resized cropping uses output 224×224 and scale $(0.7, 1.0)$; horizontal-flip probability is 0.5. Color-jitter strengths are 0.2 for brightness, contrast and saturation, and 0.05 for hue. Normalization uses the ImageNet channel statistics specified in the text.

Full training batches have shape $(64, 3, 224, 224)$. The author-reported range $[-2.12, +2.64]$ agrees with theoretical endpoints $(0 - 0.485)/0.229 = -2.118$ and $(1 - 0.406)/0.225 = +2.640$; aggregate summaries do not independently verify this batch measurement.

The 8-image training preview in Figure 3 is denormalized for interpretable RGB display.

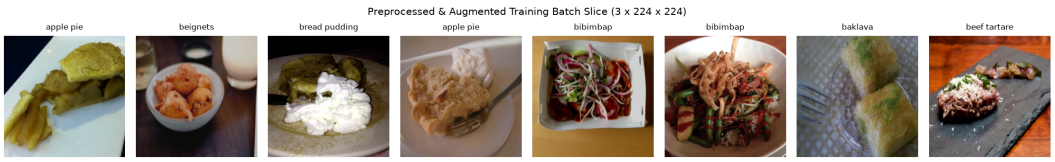


Figure 3: Example training batch slice (8 images) after random resized cropping, color jitter, and ImageNet normalization (denormalized for display).

Augmentation changes framing, orientation and appearance while retaining labels. Validation, test and no-augmentation training instead share deterministic resizing and center cropping.

3 Methodology

3.1 Pipeline Overview

Figure 4 summarizes the training, checkpoint-selection and test paths. The project implements the manifest audit, dataset wrapper, custom CNN, two-stage trainer, profiling routine and Grad-CAM hooks using PyTorch. Torchvision supplies Food-101 loading, image transformations and the pretrained ResNet-50; scikit-learn supplies stratified splitting and classification metrics. NumPy and Matplotlib support aggregation and visualization.

Automatic differentiation and optimization update the trainable parameters. The selected checkpoint is restored for test evaluation, and labels are obtained by the largest output logit without additional post-processing. Softmax probabilities support confidence-ranked examples; latency profiling and Grad-CAM are separate analyses with the measurement scopes stated below.

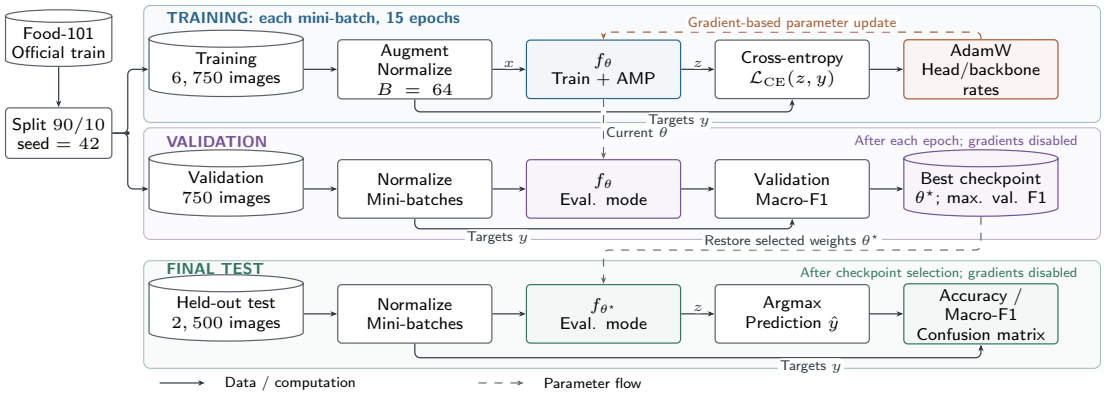


Figure 4: End-to-end Assignment 2 training, validation checkpointing, and evaluation pipeline.

Ground-truth labels enter the training loss and evaluation metrics through separate paths. Validation compares successive model states without optimizer updates, whereas test evaluation restores the retained checkpoint. Consequently, checkpoint selection depends on validation rather than test predictions. The diagram represents this information flow; it does not measure execution time or depict the separate confidence-ranking and latency procedures.

3.2 Model Architectures

3.2.1 Custom Convolutional Neural Network

The custom CNN learns directly from the 6,750 training images without external pretrained weights. Its operations and intermediate representations are specified in Table 3. The network has 1,242,090 parameters, all trainable.

Output shapes describe one image and omit the batch axis. Successive blocks increase channel width while pooling reduces spatial resolution. Adaptive averaging supplies the classifier input, separating spatial feature extraction from the final class readout.

Table 3: **Custom Convolutional Network Architecture**

Stage	Operation	Configuration	Output[a]
Block 1	Two convolution units	$3 \to 32 \to 32$	$(32, 112, 112)$
Block 2	Two convolution units	$32 \to 64 \to 64$	$(64, 56, 56)$
Block 3	Two convolution units	$64 \to 128 \to 128$	$(128, 28, 28)$
Block 4	Two convolution units	$128 \to 256 \to 256$	$(256, 14, 14)$
Aggregation	Adaptive average pooling	Output grid $(1, 1)$; flatten	256
Hidden head	Linear, BN, ReLU, dropout	$256 \to 256$; $p = 0.3$	256
Readout	Linear	$256 \to 10$	10

[a] Each convolution unit applies a 3×3 convolution (padding 1, no bias), BatchNorm2d and ReLU. Each block ends with 2×2 max pooling at stride 2. BN in the hidden head denotes BatchNorm1d. Shapes omit the batch dimension; the readout produces class logits.

Local receptive fields and shared convolutional kernels encode spatial locality. Repeated convolution and pooling construct progressively coarser feature maps, with spatial sizes $224 \to 112 \to 56 \to 28 \to 14$. Global average pooling compresses the final $(256, 14, 14)$ representation before classification, limiting the parameter cost of the dense head. Batch normalization and dropout regularize the learned representation, although their presence does not establish immunity to overfitting.

3.2.2 Pretrained ResNet-50

ResNet-50 provides a deeper residual representation initialized from Torchvision's ImageNet-1K V1 weights. Residual additions of the form $x + \mathcal{F}(x)$ facilitate gradient propagation. Table 4 identifies the backbone and replacement classification head.

Table 4: **ResNet-50 Architecture**

Stage	Operation	Configuration	Output[a]
Stem	Convolution and max pooling	7×7 stem convolution	Spatial feature maps
Backbone	Four residual stages	Bottleneck blocks; skip connections	Spatial feature maps
Aggregation	Global average pooling	Pooled backbone representation	2,048
Readout	Replacement linear head	$2048 \to 10$	10

[a] Torchvision supplies the 50-layer ImageNet-1K V1 pretrained backbone. The original $1,000$-class head is replaced by a 10-class linear layer. The resulting model contains $23,528,522$ total parameters; $20,490$ belong to the new head.

The new head contains $2,048 \times 10 + 10 = 20,490$ parameters, approximately 0.087% of the $23,528,522$-parameter model. In the frozen configuration, only these head parameters receive gradient updates. In the two-stage configuration, the same initialization is followed by head warm-up and full-backbone adaptation. Pretraining supplies a visual representation learned outside the selected food subset; the comparison with the custom CNN also changes architecture and capacity, so it does not isolate pretraining alone.

3.3 Backbone Update Policies

The learning-rate schedule in Table 5 distinguishes the three primary configurations. Head warm-up allows the new classifier to adapt before the pretrained backbone is updated. The two-stage policy then uses a lower learning rate for backbone parameters than for the classification head.

Table 5: **Training Policies and Learning Rates**

Configuration	**Epochs** 1–5	**Epochs** 6–15
Custom CNN	All parameters, 10^{-3}	All parameters, 10^{-3}
Frozen ResNet-50	Head only, 10^{-3}	Head only, 10^{-3}
Fine-tuned ResNet-50	Head only, 10^{-3}	Backbone 10^{-4}; head 5×10^{-4}

[a] All configurations use AdamW, weight decay 10^{-4} and batch size 64. The fine-tuned model unfreezes all parameters at epoch 6 and constructs a new optimizer with separate parameter groups. The ablation uses the same two-stage policy.

At epoch 6, all $23,528,522$ parameters of the fine-tuned model become trainable. Optimizer reinitialization, backbone unfreezing and learning-rate changes occur together; their individual contributions are not isolated. In the frozen configuration and warm-up stage, freezing applies to parameters. The training loop still places the full model in training mode, allowing BatchNorm running statistics to update. The frozen configuration is therefore defined by fixed backbone parameters rather than permanently fixed normalization buffers.

4 Experimental Protocol

4.1 Training Configuration and Checkpoint Selection

All main configurations use AdamW with weight decay 10^{-4}, batch size $B = 64$ and a 15-epoch schedule. Cross-entropy receives logits directly. On CUDA, training uses automatic mixed precision and gradient scaling; validation and test evaluation use the evaluation-mode forward pass without autocast. No early stopping, cosine scheduler, MixUp or CutMix is implemented. The custom CNN uses dropout $p = 0.3$ in its classifier.

After each epoch, the 750-image validation partition is evaluated. A checkpoint replaces the previous one only when validation Macro-F1 strictly improves. Both validation accuracy and Macro-F1 in the summary describe this selected epoch; they are not independent maxima. The

selected weights are restored for test evaluation. Configuration choices are specified by the implementation rather than an automated search, and prior manual tuning is not documented.

Training data are shuffled, while validation and test order are fixed. The loader defaults to 2 workers, pinned memory when CUDA is available and nonblocking device transfers. All primary models use seed 42, but a shared loader's generator can advance between runs. A fixed seed and common manifest do not establish identical minibatch order across configurations or identical behavior across software and hardware environments.

4.2 Experimental Design

The primary experiment evaluates whether full adaptation improves recognition enough to justify additional compute. The secondary experiment changes the augmentation policy. Table 6 identifies the intended factor changes, shared settings, evaluation measures and conclusion criteria.

Table 6: **Structured Experimental Design**

Element	**Primary comparison**	**Augmentation ablation**
Hypothesis	Full fine-tuning improves predictive scores over head-only training, at greater training and memory cost.	Removing augmentation permits faster training-set fitting and may change validation stability and selected-checkpoint performance.
Changed factor	ResNet-50 update policy; custom CNN provides a separate architecture reference.	Augmented versus deterministic training crops.
Shared settings	Manifest seeded 42, 224×224 inputs, augmentation, AdamW, 15 epochs, AMP and validation Macro-F1 selection.	Same manifest, ResNet-50 architecture, $5 + 10$ epochs, optimizer schedule and checkpoint criterion.
Metrics	Test accuracy and Macro-F1; validation metrics; parameters, training time, latency and peak memory.	Validation/test accuracy and Macro-F1, training time and author-supplied loss/metric trajectories.
Decision criterion	Assess predictive gains jointly with runtime, allocated memory and warmed-up inference cost.	Compare selected-checkpoint metrics and late-epoch behavior; distinguish observed differences from causal or statistical claims.

[a] The custom CNN comparison also changes architecture and capacity. The no-augmentation routine does not explicitly reseed model initialization, and shuffled-loader states can differ. These implementation limits qualify interpretation of the intended controlled comparison.

Accuracy measures the proportion of correctly classified images; Macro-F1 averages class-specific F1 scores without weighting by support. Differences describe single-run outcomes, not statistical significance. The no-augmentation routine reuses the manifest and schedule but does not explicitly reset global random state before model construction, so its results cannot isolate the effect of augmentation.

4.3 Hardware and Software Environment

The author-reported environment is a Lenovo Legion laptop with an Intel Core Ultra 9 275HX CPU, NVIDIA GeForce RTX 5060 Laptop GPU and 32 GB RAM, running WSL2 Kali Linux on Windows 11. The GPU has a nominal 8 GB capacity; the supplied report records 7.96, which the implementation calculates in GiB. Python 3.13.13 and PyTorch 2.12.1 with CUDA 13.0 (`+cu130`) are recorded in the repository metadata. All reported main training runs use the GPU.

The dependency specification gives minimum versions for Torchvision (0.15.0), NumPy (1.21.0), Matplotlib (3.5.0), scikit-learn (1.0.0) and Pillow (9.0.0). These bounds do not form a locked environment, and a captured installation record is unavailable. Deterministic CuDNN settings and a recorded seed support reconstruction but do not establish independent reproduction of the reported measurements.

4.4 Reproducibility and Measurement Protocol

The following protocol distinguishes implementation reconstruction, recorded results and the scope of computational measurements:

1. **Reconstruction and provenance:** The archived implementation provides the dataset audit, manifest and model definitions; reconstruction instructions describe training and evaluation. Aggregate results and figures are available, but original run identifiers, checkpoint hashes, retained checkpoint binaries and serialized epoch histories are not. Repository identity establishes the inspected source revision rather than the original generating run. Executing the pipeline additionally requires the Food-101 images and ImageNet pretrained weights; the retained manifest and summaries alone do not constitute a runnable data archive.
2. **Result verification:** Primary and ablation summary values are checked against the repository CSVs. The supplied epoch table and detailed loss trajectories are retained as author-supplied observations because their underlying histories are unavailable. The manuscript distinguishes these records from independently checked aggregate entries. Agreement with rounded summaries establishes manuscript consistency, not independent reproduction of the generating runs.
3. **Training and memory:** Training duration spans the full epoch loop, including validation and checkpoint writing, followed by synchronization. Peak memory is the CUDA allocator's maximum allocated bytes after the per-run reset, divided by 1024^2 and reported here in MiB. Earlier trained models can remain resident, so these peaks are not isolated model-only memory requirements or total device usage.
4. **Test traversal and latency:** Test traversal includes data loading, transfers, prediction and collection of outputs for metrics. The separate latency benchmark uses a fixed device-resident batch of 64 images, 5 warm-up passes and 20 timed forward passes, with synchronization around timing and no autocast. Its per-image value is a batch average, not measured single-image latency. The timed forwards exclude data loading and preprocessing.
5. **Qualitative evidence and disclosure:** Confidence-ranked examples and Grad-CAM explain selected predictions qualitatively. The repository's AI-use record contains named author declarations and representative prompts. These records are not independent evidence that training or verification was repeated during this report revision.

5 Results and Discussion

5.1 Training Dynamics

Learning curves provide context for the selected checkpoints. Figure 5 compares the three primary configurations over 15 epochs and marks the stage boundary after epoch 5.

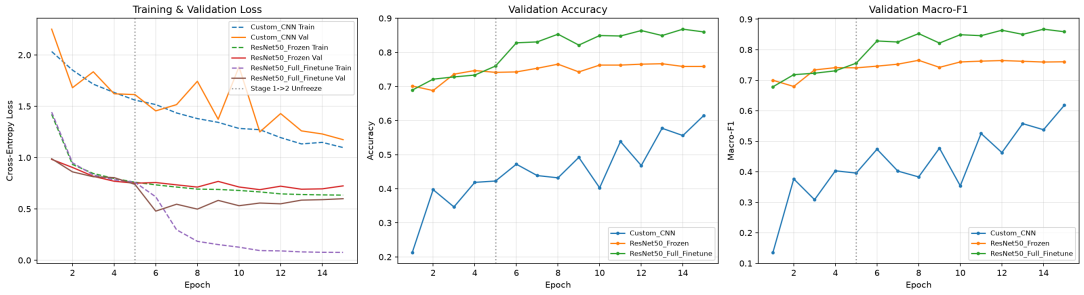


Figure 5: Training and validation cross-entropy loss, validation accuracy, and validation Macro-F1 across 15 epochs. The dotted line after epoch 5 marks the stage transition for full fine-tuning.

The curves suggest gradual scratch learning, an early plateau for the frozen model, and improvement after full fine-tuning. They do not isolate architectural, capacity or optimization effects.

Reported epoch-wise Macro-F1 values, including the ablation, are listed in Table 7.

Table 7: **Reported Epoch-wise Validation Macro-F1**

Epoch	Custom CNN	Frozen ResNet-50	Fine-tuned ResNet-50	No augmentation
1	0.1358	0.6999	0.6786	0.6578
2	0.3765	0.6796	0.7183	0.7372
3	0.3085	0.7339	0.7230	0.7375
4	0.4032	0.7412	0.7308	0.7468
5	0.3958	0.7407	0.7560	0.7467
Epoch 6: backbone unfreezing for both fine-tuned configurations				
6	0.4740	0.7462	0.8287	0.8144
7	0.4023	0.7527	0.8253	0.8395
8	0.3831	**0.7653**	0.8529	0.8504
9	0.4774	0.7422	0.8218	0.8620
10	0.3539	0.7599	0.8492	**0.8721**
11	0.5255	0.7626	0.8460	0.8472

Continued on next page

Table 7: **Reported Epoch-wise Validation Macro-F1** (Continued)

Epoch	Custom CNN	Frozen ResNet-50	Fine-tuned ResNet-50	No augmentation
12	0.4629	0.7646	0.8639	0.7907
13	0.5578	0.7622	0.8503	0.8040
14	0.5378	0.7597	**0.8673**	0.7892
15	**0.6180**	0.7605	0.8589	0.8423

[a] Epoch values come from the supplied author report; serialized histories are unavailable. Bold marks the reported selected epoch. Main selected values agree with the aggregate CSVs.

Learning from scratch. Starting from random weights, the CNN reaches 21.33% validation accuracy and 0.1358 Macro-F1 at epoch 1. Its selected epoch 15 records 61.47% accuracy, 0.6180 Macro-F1 and training loss 1.0981. These author-supplied dynamics suggest continued learning; longer schedules were not tested.

Head-only transfer. The frozen model reaches 70.13% validation accuracy after one epoch (approximately 15 s in the supplied record), above the CNN's final accuracy. Its selected checkpoint is epoch 8, with 76.53% accuracy and 0.7653 Macro-F1. Reported late accuracy spans 74.2%–76.7%, with final training loss 0.6344. Individual epoch timings were not retained.

Full fine-tuning. Epoch 5 records 76.00% validation accuracy and 0.7560 Macro-F1. At unfreezing (epoch 6), validation loss falls from 0.7394 to 0.4786 and accuracy rises to 82.80% (+6.80 percentage points). The selected epoch 14 gives 86.80% accuracy and 0.8673 Macro-F1. This change accompanies both unfreezing and optimizer reset, whose effects are not isolated.

5.2 Food-101 Benchmark

Predictive quality and compute requirements are considered together in Table 8. The comparison uses the same 2,500-image official test partition, with checkpoints selected from validation Macro-F1.

Table 8: **Main Quantitative and Compute-Cost Comparison** on the ten-class Food-101 subset (seed 42).

Metric[a]	Custom CNN	Frozen ResNet-50	Fine-tuned ResNet-50
↓ Total parameters	**1,242,090**	23,528,522	23,528,522
↓ Trainable parameters	1,242,090	**20,490**	23,528,522
Selected epoch	15	8	14
↑ Val accuracy	0.6147	0.7653	**0.8680**
↑ Val Macro-F1	0.6180	0.7653	**0.8673**
↑ Test accuracy	0.6336	0.8024	**0.8880**

Continued on next page

Table 8: **Main Quantitative and Compute-Cost Comparison** on the ten-class Food-101 subset (seed 42). (Continued)

Metric[a]	Custom CNN	Frozen ResNet-50	Fine-tuned ResNet-50
↑ Test Macro-F1	0.6335	0.8022	**0.8877**
↓ Train time (s)	**219.32**	223.42	267.71
↓ Test traversal (s)	**2.88**	3.95	4.12
↓ Latency (ms/batch)	**57.22**	91.27	91.93
↓ Latency (ms/image)	**0.894**	1.426	1.436
↓ Peak memory (MiB)	2,093.38	**855.49**	3,218.35

[a] Primary runs use 15 epochs, seed 42 and AMP. Validation metrics describe the checkpoint selected by validation Macro-F1. Trainable parameters are the maximum across stages. Train time includes validation and checkpoint saving.

[b] Traversal includes loading, transfers and metric collection. Latency averages 20 timed passes after 5 warm-up passes on resident batch-64 inputs, without autocast; per-image values are batch averages.

[c] Memory is peak tensor allocation in MiB (1024^2 bytes); the CSV uses an MB label. Prior models may remain resident. Arrows indicate preferred directions; bold marks the best reported value per row.

Two-stage full fine-tuning gives the highest primary-comparison test accuracy (88.80%) and Macro-F1 (0.8877), compared with 80.24% and 0.8022 for the frozen model and 63.36% and 0.6335 for the custom CNN. The result favors adaptation within this main comparison, while the lowest runtime, smallest trainable parameter count and lowest allocated memory occur in different configurations.

5.2.1 Architectural and Computational Trade-offs

Pretrained representations. The frozen model exceeds the custom CNN by +16.88 percentage points of test accuracy and +0.1687 Macro-F1, reducing errors from 916 to 494 among 2,500 test images. Only 20,490 head parameters receive gradient updates, whereas the custom model learns all parameters from the selected training set. The result is consistent with useful pretrained visual features, but differences in backbone architecture, total capacity and normalization behavior prevent attributing the entire gain to pretraining alone.

Full adaptation. Fine-tuning improves on the frozen model by +8.56 percentage points of test accuracy and +0.0855 Macro-F1. The total error count decreases from 494 to 280, a reduction of 214 (43.32%). Updating the representation is a plausible explanation for better separation of related food classes, but the results do not establish particular learned texture detectors as the cause.

Validation and test differences. Test accuracy exceeds selected-checkpoint validation accuracy in all three configurations: 63.36% versus 61.47%, 80.24% versus 76.53%, and 88.80% versus 86.80%. These differences are roughly +2.0 to +3.7 percentage points. The validation split derives from the intentionally uncleaned training release, while official test images were manually reviewed. This distinction is a plausible contributor, not an isolated causal explanation for each gap.

Memory and training time. Peak allocated memory is lowest for the frozen model (855.49 MiB), compared with 2,093.38 MiB for the custom CNN and 3,218.35 MiB for full fine-tuning. The latter is 3.76× the frozen value under the recorded profiling procedure. Freezing parameters reduces backward-state requirements; full adaptation introduces backbone gradients and optimizer state. The custom CNN also retains large early feature maps for backpropagation. These mechanisms contextualize the peaks without separating their contributions from previously resident models.

Fine-tuning takes 267.71 s (approximately 4.46 min), 19.82% longer than the frozen run's 223.42 s. The first 5 epochs update only the head, and all primary training uses AMP. Since no AMP-off run or stage-isolated timing is supplied, the observed training time does not quantify the independent benefit of Tensor Core execution or mixed precision.

Inference cost. The custom CNN is fastest at 57.22 ms per batch, or 0.894 ms per image. The frozen and fine-tuned ResNet-50 models require 91.27 and 91.93 ms per batch (1.426 and 1.436 ms per image). These batched timings correspond approximately to 1,118 images/s for the CNN and about 699 images/s for the ResNet-50 variants, around 1.61× slower. The ResNet variants share a forward architecture and have similar measured latency, but the recorded timings are not identical and no repeated-run variability is available.

5.3 Augmentation Ablation

The ablation replaces random resized crops, horizontal flips and color jitter with deterministic resizing to 256 and center cropping to 224. The backbone, two-stage 5 + 10-epoch schedule, optimizer learning rates and manifest are retained. Table 9 reports the available aggregate outcomes.

Table 9: **Augmentation Ablation: Selected-Checkpoint Results**

Metric[a]	With augmentation	Without augmentation
Total parameters	23,528,522	23,528,522
Trainable parameters	23,528,522	23,528,522
↑ Val accuracy	0.8680	**0.8720**
↑ Val Macro-F1	0.8673	**0.8721**
↑ Test accuracy	0.8880	**0.9024**
↑ Test Macro-F1	0.8877	**0.9021**
↓ Train time (s)	267.71	**229.87**

[a] Values are taken from the repository augmentation-ablation summary. Validation metrics correspond to the selected validation-Macro-F1 checkpoint. Arrows indicate preferred directions; bold marks preferred measured values. Ablation-specific peak memory and latency are not exported.

The run without augmentation attains the higher selected-checkpoint validation scores (0.8720 accuracy, 0.8721 Macro-F1) and test scores (0.9024 accuracy, 0.9021 Macro-F1), compared with 0.8680, 0.8673, 0.8880 and 0.8877 with augmentation. Its training time is also shorter (229.87 s versus 267.71 s). The recorded comparison therefore does not establish augmentation as the

best-performing choice. Initialization and loader-state limitations, together with the single-run design, prevent a general causal conclusion that augmentation is harmful.

The selected-checkpoint result does not fully describe the reported optimization trajectory. Table 10 retains the supplied loss and late-epoch diagnostics so that transient performance and checkpoint selection can be distinguished.

Table 10: **Reported Augmentation Ablation Diagnostics**

Metric[a]	With augmentation	Without augmentation
Selected epoch	14	10
↓ Epoch 10 train loss	0.1273	**0.0041**
↓ Epoch 12 val loss	**0.5498**	0.9607
↑ Epoch 12 val accuracy	**0.8640**	0.7907
↑ Selected val accuracy	0.8680	**0.8720**
↑ Selected val Macro-F1	0.8673	**0.8721**
↑ Final-epoch val Macro-F1	**0.8589**	0.8423

[a] Epoch-specific losses, accuracies, final-epoch values and selected epochs are author-supplied diagnostics retained from the original report, not independently recovered histories. Best validation accuracy and Macro-F1 agree with the repository ablation summary. A lower training loss alone does not establish better generalization.

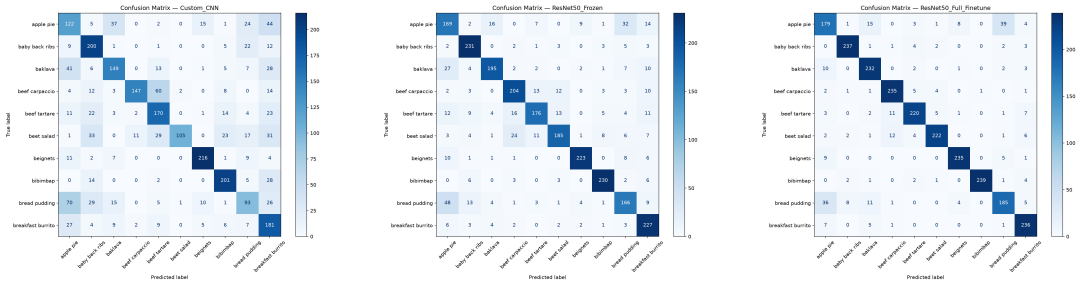
Rapid training-set fitting. After unfreezing at epoch 6, the supplied no-augmentation record shows training loss falling from 0.5566 to 0.0947 at epoch 7, 0.0159 at epoch 8, and 0.0041 at epoch 10, compared with 0.1273 with augmentation. This is consistent with faster fitting of deterministic training crops; it does not by itself identify memorized labels or prove a particular confidence level.

Late-epoch variation. The supplied record gives a no-augmentation validation loss of 0.9607 at epoch 12, compared with 0.4508 at epoch 8. Validation accuracy falls from 87.20% at epoch 10 to 79.07% at epoch 12, a −8.13-percentage-point change. The augmented run's reported epoch-12 loss is 0.5498 and accuracy is 0.8640. The author-supplied later-epoch accuracy range of 84.80%–86.80% and final Macro-F1 values (0.8589 with augmentation, 0.8423 without) suggest differences in late behavior, while the CSV test scores still favor no augmentation. None of these observations demonstrates label noise as the cause of the fluctuation.

5.4 Qualitative Results and Interpretability

Class-level errors complement aggregate scores: the confusion matrices in Figure 6 compare correct assignments with the classes receiving misclassified images.

Rows denote true labels and columns predicted labels. With the same class order and test membership, diagonal and off-diagonal counts permit direct comparison of correct assignments and confusion patterns across models.



(a) Custom CNN (63.36%) (b) Frozen ResNet-50 (80.24%) (c) Fine-tuned ResNet-50 (88.80%)

Figure 6: Comparison of test set confusion matrices across the three primary configurations (2,500 test images, 250 per class).

Fine-tuning improves the aggregate result while leaving visible confusions between related foods. Pastry categories and visually similar red dishes remain relevant failure cases. The matrices locate these errors but do not determine whether texture, crop framing, illumination or training annotations caused any particular prediction.

At the image level, Figure 7 separates high-confidence correct predictions, low-confidence correct predictions and high-confidence errors. The implementation ranks correct examples by the winning-class probability, rather than by an explicitly computed top-two probability margin.



Figure 7: Three-tier qualitative analysis for fine-tuned ResNet-50: high-confidence correct predictions (top), low-confidence correct predictions (middle), and high-confidence failures (bottom).

The displayed failures include apple pie predicted as bread pudding, baklava predicted as apple pie, beet salad predicted as beef carpaccio, and breakfast burrito predicted as apple pie. Correct and incorrect examples can both receive high displayed confidence, emphasizing that these selected probabilities do not constitute a calibration evaluation. The selected grid does not estimate the frequency of each visual failure mode.

Grad-CAM offers a separate view of the spatial responses associated with predicted labels. Figure 8 compares the custom CNN's final convolutional block with the final residual block of each ResNet-50 variant on the same test examples. The method spatially averages gradients, weights the captured feature maps, applies ReLU and upsamples the result. Enabling input gradients permits this calculation even with frozen backbone parameters.

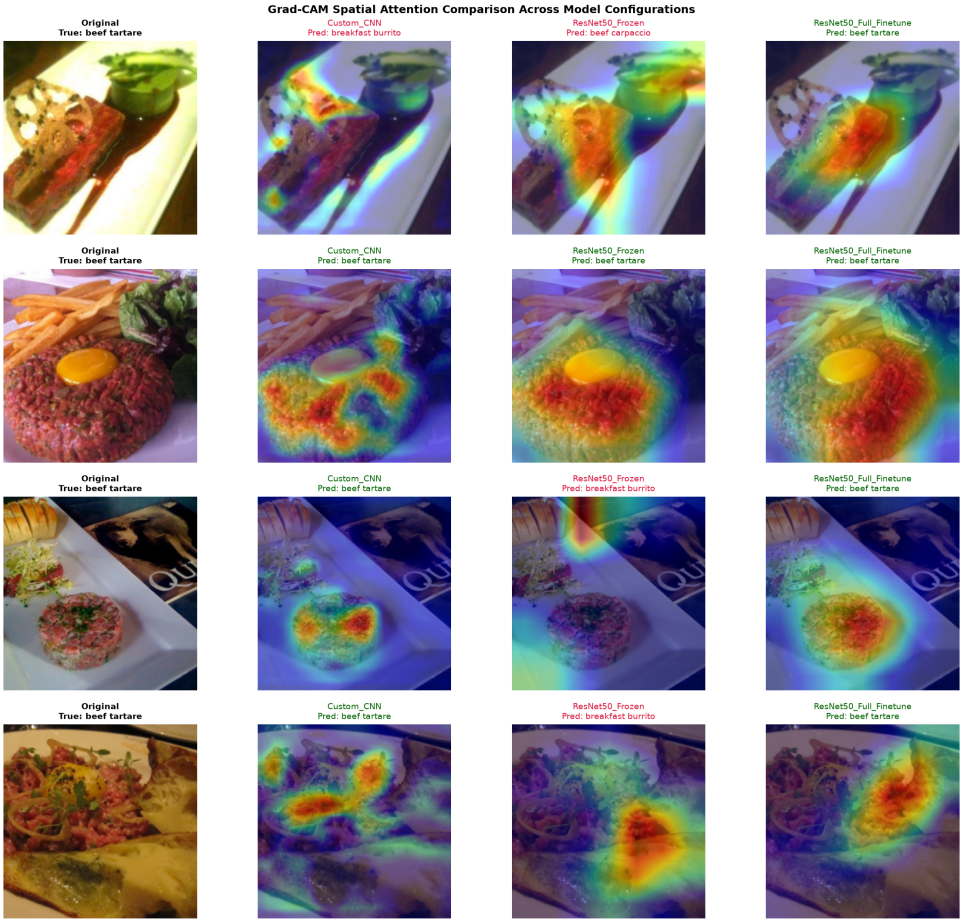


Figure 8: Grad-CAM overlays for the custom CNN and frozen and fine-tuned ResNet-50 models on shared test images.

All four displayed source images are labeled beef tartare. The fine-tuned model predicts their labels correctly and often emphasizes the central food region; the other configurations show different activation patterns and some incorrect predictions. Grad-CAM targets each model's own predicted class, so maps for incorrect predictions can be conditioned on different classes.

These examples do not establish dataset-wide background suppression, causal feature use or sensitivity to pastry structures that are absent from the displayed set.

5.5 Error Analysis

The fine-tuned primary model makes 280 test errors, compared with 494 for the frozen model. Table 11 supplies representative pair counts and rates to ground the qualitative taxonomy in recorded predictions.

Table 11: **Representative Confusion Pairs**

True class	Predicted class	Count	Rate (%)[a]
Apple pie	Bread pudding	39	15.60
Bread pudding	Apple pie	36	14.40
Beet salad	Beef carpaccio	12	4.80
Beef tartare	Beef carpaccio	11	4.40
Bread pudding	Baby back ribs	8	3.20
Beef tartare	Breakfast burrito	7	2.80

[a] Counts and class-conditional rates are read from the fine-tuned model's exported error-pair summary. Each true class contains 250 test images. The displayed pairs are illustrative, not an exhaustive partition of the 280 errors; category labels describe hypothesized mechanisms.

The largest displayed pair counts involve apple pie and bread pudding, while other examples span red-food similarity, mixed ingredients and dark-glazed appearances. These pair frequencies identify where errors occur. They are not validated measurements of the proposed visual causes.

The five categories below organize interpretation and future tests. The exported category assignment is a heuristic based on class pairs, not an independently annotated causal taxonomy:

1. **Red-dish visual similarity:** Beef carpaccio, beef tartare and beet salad can share crimson colors, greens and garnishes. Recorded beet-salad-to-carpaccio and tartare-to-carpaccio confusions suggest this ambiguity, although texture and shape may distinguish them. No feature probe establishes reliance on raw color histograms or explains individual failures.
2. **Baked-dessert and pastry ambiguity:** Apple pie, baklava, bread pudding and beignets can share browned crusts, syrup, powdered sugar or cream. Apple pie and bread pudding are prominent bidirectional confusions. Containers and toppings may obscure pastry structure; higher-resolution observations could test whether additional detail helps.
3. **Mixed ingredients and occlusion:** Burrito cross-sections and mixed bowls can expose overlapping ingredients and lose characteristic arrangement cues. The recorded tartare-to-burrito confusion and the displayed burrito failure motivate this hypothesis. Neither the class-pair label nor a selected image measures the independent effect of occlusion.
4. **Lighting and close framing:** Dark sauces and shadows may reduce contrast between foods. The exported summary includes bread pudding predicted as baby back ribs, rather than establishing the reverse direction as a dominant error. Controlled illumination or crop perturbations would be needed to isolate this explanation.

5. **Presentation variation and suspected label noise:** Multi-dish settings and unusual serving styles can complicate recognition. Training-label noise is documented for Food-101, but its effect on these particular decision boundaries is not measured. Grad-CAM alone cannot confirm noisy labels as the cause of a test error; no reliable category-wide causal frequency is available.

Future experiments could test label smoothing ($\epsilon = 0.1$) or symmetric cross-entropy, cosine learning-rate decay during the second stage, and evaluation at 384×384. These are proposed interventions rather than improvements demonstrated by the current experiments.

5.6 Limitations

The results cover ten predefined classes, one seed (42), one pretrained backbone family and 224×224 inputs. Repeated runs, for example 3–5 seeds with mean $\pm$ standard deviation, would be needed to characterize variation. No cosine schedule or MixUp/CutMix comparison is supplied, and the augmentation result should not be generalized beyond the recorded setting.

The available CSVs verify aggregate consistency, not original run provenance. Serialized histories, checkpoint hashes, original training-run identifiers and a locked environment are unavailable. Epoch-specific results retained from the supplied report remain author-provided. Perceptual-hash matches span partitions; frozen-backbone normalization buffers can update; ablation initialization is not explicitly reseeded; and earlier models may remain resident during memory measurement. These limits qualify claims of strict leakage prevention, isolated factor effects and model-only GPU memory requirements.

Evaluation also assumes that every input belongs to one of the selected categories. The study does not measure rejection of unfamiliar dishes, performance under changes in capture conditions, or calibration of predicted probabilities. Consequently, confidence-ranked examples do not establish that the models can identify unsupported inputs or reliably quantify uncertainty in deployment.

6 Conclusion

Within the primary comparison, two-stage full fine-tuning improves test accuracy to 88.80% and Macro-F1 to 0.8877, exceeding the custom CNN by +25.44 percentage points of accuracy and the frozen ResNet-50 by +8.56 points. Relative to the frozen model, it reduces the total error count by 43.32% while taking 19.82% longer to train and reaching 3.76× the recorded peak allocated memory. The ResNet variants have similar, separately measured inference latency rather than an established zero-cost equivalence.

The augmentation ablation changes the recommendation: the no-augmentation run records the highest available test accuracy (90.24%) and Macro-F1 (0.9021), despite the author's reported late-epoch instability. Full fine-tuning is therefore supported as a promising adaptation strategy in this subset, while augmentation is not established as the optimal policy. Selection should consider predictive quality, compute constraints and the limits of this single-run comparison; no universal ranking or statistically significant causal effect is claimed.

AI Usage Disclosure

The group declares the following AI assistance during Assignment 2. Member-specific prompts, affected components and author-declared verification records are linked in the repository disclosure.

- **Tool:** Google Gemini (2026 release). **Used by:** All 4 group members during M1, M2 and M3, as reported by the authors.
- **Scope of assistance:** Hash-audit and CSV utilities, EDA and prediction-grid formatting, profiling syntax, Grad-CAM overlays, and LaTeX typesetting.
- **Student verification:** The authors report local verification against library documentation and experiment outputs. Aggregate values have been checked against repository CSVs; retained epoch diagnostics are author-supplied. No training was repeated during this revision.
- **Report editing:** OpenAI Codex helped maintain consistent template, table and figure formatting, supported academic editing, and assisted with source/CSV checks and PDF verification. Editing requests and checks accompany the report source; no new experimental data were generated.